

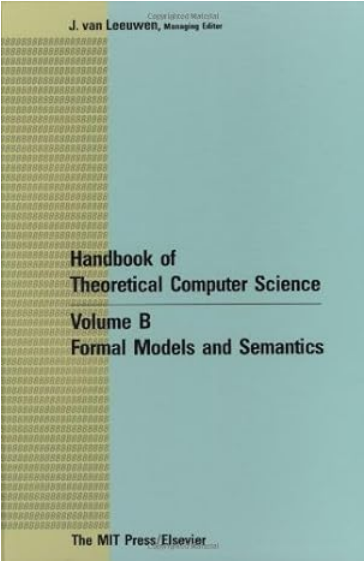
SI1001 Teoría de la Computación

Lambda cálculo

Andrés Sicard Ramírez

Universidad EAFIT

Semestre 2025-2



CHAPTER 7

Functional Programming and Lambda Calculus

H. P. BARENDREGT

Faculty of Mathematics and Computer Science, Catholic University Nijmegen,
Toernooiveld 1, 6525 ED Nijmegen, Netherlands

Contents

1. The functional computation model	323
2. Lambda calculus	325
3. Semantics	337
4. Extending the language	347
5. The theory of combinators and implementation issues	355
Acknowledgment	360
References	360

Convenciones

- El texto guía para estas diapositivas es (Barendregt 1992a).
- Los números asignados a las definiciones, ejemplos, ejercicios, figuras, páginas, secciones y teoremas en estas diapositivas corresponden a los números asignados en el texto guía.
- Los números naturales incluyen el cero, es decir, $\mathbb{N} = \{0, 1, 2, \dots\}$.
- El conjunto potencia de un conjunto A es denotado $\mathcal{P}A$.
- Los términos «definición inductiva» y «definición recursiva» se usan como sinónimos.
- En las definiciones inductivas se sobreentiende que el conjunto definido es el menor conjunto que satisface las condiciones.

Preliminares

Agradecimientos

Agradezco a mis colegas René Alejandro Londoño Cano y Sergio Steven Ramírez Rico por señalarme algunas correcciones en una versión anterior de estas diapositivas.

Esquema de la presentación

- Introducción
- Descripción formal del lambda cálculo
- El lambda cálculo como una teoría formal
- Combinadores
- Combinadores de punto fijo
- Codificación de Church
- Codificación de Barendregt
- Representación de las funciones computables
- Referencias

Tema

- **Introducción**
- Descripción formal del lambda cálculo
- El lambda cálculo como una teoría formal
- Combinadores
- Combinadores de punto fijo
- Codificación de Church
- Codificación de Barendregt
- Representación de las funciones computables
- Referencias

Introducción

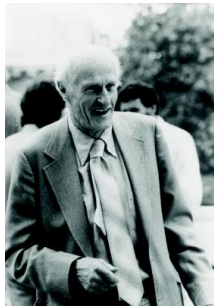
Alonzo Church (1903 – 1995)^a



^aFiguras tomadas de History of computers, Wikipedia y MacTutor History of Mathematics.

Introducción

Stephen Cole Kleene (1909 – 1994)^a



John Barkley Rosser (1907 – 1989)^b



^aFiguras tomadas de MacTutor History of Mathematics y Oberwolfach.

^bFigura tomada de la John Simon Guggenheim Memorial Foundation.

Introducción

- Un sistema formal creado por Alonzo Church alrededor de 1930 y sus estudiantes Stephen Kleene y J. Barkley Rosser.
- El objetivo inicial fue usar el lambda cálculo para las fundaciones de la matemáticas.
- Empleado para estudiar funciones y recursividad.
- Modelo de computabilidad.
- Base de los lenguajes de programación funcionales (Lisp, Haskell, ML, Scheme, etc.)
- Notación (*p. ej.* funciones anónimas y curryficación).

Introducción

Bibliografía

- Artículos introductorios: (Barendregt 1992b, § 2) y (Barendregt y Barendsen 2000).
- Una presentación sucinta de la teoría: (Seldin 2009, Apéndice A).
- Una presentación de los desarrollos actuales: (Cardone e Hindley 2009)
- Un artículo acerca de la historia del lambda cálculo: (Rosser 1984). Véase también (Cardone e Hindley 2009) y (Seldin 2009).
- Libro de texto: (Hindley y Seldin 2008).
- La «biblia»: (Barendregt 2004).

Introducción

Aplicación, λ -abstracción y β -conversión (informalmente)

En el tablero.

Introducción

Definición

Una función es de **orden superior** sii

- (i) recibe (al menos) una función como un argumento o
- (ii) retorna una función como resultado.

Introducción

Definición

Una función es de **orden superior** sii

- (i) recibe (al menos) una función como un argumento o
- (ii) retorna una función como resultado.

Ejemplos

En el tablero.

Introducción

Definición

Una función es de **orden superior** sii

- (i) recibe (al menos) una función como un argumento o
- (ii) retorna una función como resultado.

Ejemplos

En el tablero.

Observación

Las funciones de orden superior son usualmente llamadas «funcionales» en matemáticas.

Introducción

Funciones de varias variables

Funciones de varias variables pueden ser representadas por funciones de orden superior y usos repetidos de aplicación. Esta técnica fue propuesta por Schönfinkel, pero es llamada **curryficación** debido a que fue popularizada por Haskell Curry.

Introducción

Ejemplo (curryficación)

Sea $g : X \times Y \rightarrow Z$ una función de dos variables. Podemos definir dos funciones f_x y f por:

$$f_x : Y \rightarrow Z$$

$$f_x = \lambda y. g(x, y),$$

$$f : X \rightarrow (Y \rightarrow Z)$$

$$f = \lambda x. f_x.$$

Entonces $(f\ x)\ y = f_x\ y = g(x, y)$. Es decir, la función de dos variables

$$g : X \times Y \rightarrow Z$$

es representada por la función curryficada de orden superior.

$$f : X \rightarrow (Y \rightarrow Z).$$

Tema

- Introducción
- **Descripción formal del lambda cálculo**
- El lambda cálculo como una teoría formal
- Combinadores
- Combinadores de punto fijo
- Codificación de Church
- Codificación de Barendregt
- Representación de las funciones computables
- Referencias

Lambda términos

Definición 2.1.1

Sea V un conjunto enumerable de variables. El conjunto de **lambda términos** (λ -términos), denotado por Λ , es inductivamente definido por:

$x \in V \Rightarrow x \in \Lambda$ (variable)

$M, N \in \Lambda \Rightarrow (M N) \in \Lambda$ (aplicación)

$M \in \Lambda, x \in V \Rightarrow (\lambda x.M) \in \Lambda$ (λ -abstracción)

Lambda términos

Definición 2.1.1

Sea V un conjunto enumerable de variables. El conjunto de **lambda términos** (λ -términos), denotado por Λ , es inductivamente definido por:^a

$$x \in V \Rightarrow x \in \Lambda \quad \text{(variable)}$$

$$M, N \in \Lambda \Rightarrow (M N) \in \Lambda \quad \text{(aplicación)}$$

$$M \in \Lambda, x \in V \Rightarrow (\lambda x.M) \in \Lambda \quad \text{(\lambda-abstracción)}$$

^aLa definición en el texto guía incluye constantes las cuales no serán usadas en el curso.

Lambda términos

Ejemplos

En el tablero.

Lambda términos

Definición

EL conjunto de λ -términos Λ también es definido frecuentemente empleando reglas de inferencia. Sea V un conjunto enumerable de variables, entonces:

$$\frac{x \in V}{x \in \Lambda} \text{ (variable)}$$

$$\frac{M \in \Lambda \quad N \in \Lambda}{(M N) \in \Lambda} \text{ (aplicación)}$$

$$\frac{M \in \Lambda \quad x \in V}{(\lambda x.M) \in \Lambda} \text{ (\lambda-abstracción)}$$

Lambda términos

Observación

Usualmente, el conjunto de λ -términos Λ es introducido por una gramática abstracta.^a

$\Lambda \ni t ::= x$	(variable)
$\quad \quad \ t \ t$	(aplicación)
$\quad \quad \ \lambda x. t$	(λ -abstracción)

^aVéase, *p. ej.* (Pierce 2002).

Lambda términos

Convenciones

- Las variables serán denotadas por letras minúsculas $x, y, z, \dots$
- Los λ -términos serán denotados por $L, M, N, \dots$

Ligadura de variables

Definición

La ocurrencia de una variable x en un λ -término M es **libre** si x **no está** en el alcance de λx . En caso contrario, la ocurrencia de la variable x está **ligada** por la abstracción λx .

Ligadura de variables

Definición

La ocurrencia de una variable x en un λ -término M es **libre** si x **no está** en el alcance de λx . En caso contrario, la ocurrencia de la variable x está **ligada** por la abstracción λx .

Ejemplos

En el tablero.

Ligadura de variables

Definición 2.1.4.(i)

El **conjunto de variables libres** de un λ -término M , denotado por $FV(M)$, es definido recursivamente por:

$$FV : \Lambda \rightarrow \mathcal{P}V$$

$$FV(x) = \{x\},$$

$$FV((M\ N)) = FV(M) \cup FV(N),$$

$$FV((\lambda x.M)) = FV(M) - \{x\}.$$

Alfa congruencia

Descripción

Los λ -términos que difieren solamente en los nombres de las variables **ligadas** son considerados los mismos. En este caso se dice que los λ -términos son **alfa congruentes** (α -congruentes).

Alfa congruencia

Descripción

Los λ -términos que difieren solamente en los nombres de las variables **ligadas** son considerados los mismos. En este caso se dice que los λ -términos son **alfa congruentes** (α -congruentes).

Ejemplos

En el tablero.

Identidad sintáctica

Notación

La expresión $M \equiv N$ denota que M y N son el mismo λ -término (identidad sintáctica).

Identidad sintáctica

Notación

La expresión $M \equiv N$ denota que M y N son el mismo λ -término (identidad sintáctica).

Convención

Siguiendo el texto guía, sintácticamente identificamos λ -términos que son α -congruentes, por lo tanto, $M \equiv N$ denota que M y N son el mismo λ -término o que son α -congruentes.

Identidad sintáctica

Notación

La expresión $M \equiv N$ denota que M y N son el mismo λ -término (identidad sintáctica).

Convención

Siguiendo el texto guía, sintácticamente identificamos λ -términos que son α -congruentes, por lo tanto, $M \equiv N$ denota que M y N son el mismo λ -término o que son α -congruentes.

Ejemplos

En el tablero.

Convenciones en la escritura de lambda términos

Convenciones

- Los paréntesis más externos no son escritos.
- La aplicación tiene mayor precedencia que la λ -abstracción, es decir,

$\lambda x. M N$ representa $(\lambda x. (M N))$.

- La aplicación asocia a la izquierda, es decir,

$M N_1 N_2 \dots N_k$ representa $(\dots ((M N_1) N_2) \dots N_k)$.

- La λ -abstracción asocia a la derecha, es decir,

$\lambda x_1. \lambda x_2. \dots \lambda x_n. M$ representa $(\lambda x_1. (\lambda x_2. (\dots (\lambda x_n. M) \dots)))$.

- Eliminación de λ s, es decir,

$\lambda x_1 x_2 \dots x_n. M$ representa $\lambda x_1. \lambda x_2. \dots \lambda x_n. M$

Convenciones en la escritura de lambda términos

Ejemplo

$$(\lambda x y z. x z (y z)) u v w \equiv (((\lambda x. (\lambda y. (\lambda z. ((x z) (y z)))))u)v)w).$$

Sustitución

Observación

Para simplificar la definición de la operación de sustitución, de ahora en adelante vamos a seguir la siguiente convención para la escritura de los λ -términos.

Sustitución

Observación

Para simplificar la definición de la operación de sustitución, de ahora en adelante vamos a seguir la siguiente convención para la escritura de los λ -términos.

Convención de Ottmann para la escritura de variables

En cualquier contexto (definiciones, teoremas, demostraciones, etc.) los λ -términos se escriben tal que sus variables ligadas sean diferentes a las variables libres (Barendregt 2004, pág. 26)

Sustitución

Observación

Para simplificar la definición de la operación de sustitución, de ahora en adelante vamos a seguir la siguiente convención para la escritura de los λ -términos.

Convención de Ottmann para la escritura de variables

En cualquier contexto (definiciones, teoremas, demostraciones, etc.) los λ -términos se escriben tal que sus variables ligadas sean diferentes a las variables libres (Barendregt 2004, pág. 26)

Ejemplos

En el tablero.

Sustitución

Definición

El resultado de **sustituir** cada ocurrencia libre de una variable x por un λ -término N en un λ -término M , denotado $M[x := N]$, está definido por (Barendregt 2004, Definición 2.1.15):

$$x[x := N] \equiv N;$$

$$y[x := N] \equiv y, \text{ si } x \neq y;$$

$$(\lambda y.M)[x := N] \equiv \lambda y.(M[x := N]);$$

$$(M_1 M_2)[x := N] \equiv (M_1[x := N]) (M_2[x := N]).$$

Sustitución

Definición

El resultado de **sustituir** cada ocurrencia libre de una variable x por un λ -término N en un λ -término M , denotado $M[x := N]$, está definido por (Barendregt 2004, Definición 2.1.15):

$$\begin{aligned}x[x := N] &\equiv N; \\y[x := N] &\equiv y, \text{ si } x \neq y; \\(\lambda y.M)[x := N] &\equiv \lambda y.(M[x := N]); \\(M_1 M_2)[x := N] &\equiv (M_1[x := N]) (M_2[x := N]).\end{aligned}$$

Ejemplos

En el tablero.

Beta conversión

Definición

La relación binaria de **beta conversión** (β -conversión) entre λ -términos está definida por:

$$(\lambda x.M) N =_{\beta} M[x := N].$$

Beta conversión

Definición

La relación binaria de **beta conversión** (β -conversión) entre λ -términos está definida por:

$$(\lambda x.M) N =_{\beta} M[x := N].$$

Definición

Un **beta redex** (β -redex) es un λ -término sobre el cual se puede aplicar una β -conversión. Es decir, un β -redex es un λ -término de la forma $(\lambda x.M) N$.

Beta conversión

Definición

La relación binaria de **beta conversión** (β -conversión) entre λ -términos está definida por:

$$(\lambda x.M) N =_{\beta} M[x := N].$$

Definición

Un **beta redex** (β -redex) es un λ -término sobre el cual se puede aplicar una β -conversión. Es decir, un β -redex es un λ -término de la forma $(\lambda x.M) N$.

Ejemplos

En el tablero.

Tema

- Introducción
- Descripción formal del lambda cálculo
- **El lambda cálculo como una teoría formal**
- Combinadores
- Combinadores de punto fijo
- Codificación de Church
- Codificación de Barendregt
- Representación de las funciones computables
- Referencias

La teoría λ

Introducción

- «*The principal object of study in the λ -calculus is the set of λ -terms modulo convertibility.*» (Barendregt 2004, pág. 22).
- La relación de convertibilidad, denotada $=_\beta$, es definida por la teoría λ .

La teoría λ

Definición

Los **términos** de la teoría λ son los λ -términos.

Definición

Las **fórmulas** de la teoría λ son de la forma $M =_{\beta} N$, donde M, N son λ -términos.

Definición 2.1.5

Sean L, M, N tres λ -términos y sea x una variable. La relación de convertibilidad $=_\beta$ es definida inductivamente por:

$$(\lambda x.M) N =_\beta M[x := N] \quad (\text{axioma } \beta)$$

$$(\text{relación de equivalencia}) \left\{ \begin{array}{ll} M =_\beta M & (\text{refl}) \\ M =_\beta N \Rightarrow N =_\beta M & (\text{sim}) \\ M =_\beta N, N =_\beta L \Rightarrow M =_\beta L & (\text{trans}) \end{array} \right.$$

$$(\text{relación compatible}) \left\{ \begin{array}{ll} M =_\beta N \Rightarrow M L =_\beta N L & (\text{appd}) \\ M =_\beta N \Rightarrow L M =_\beta L N & (\text{appi}) \\ M =_\beta N \Rightarrow \lambda x.M =_\beta \lambda x.N & (\text{abst}) \end{array} \right.$$

La teoría λ

Definición

Sean L, M, N tres λ -términos y sea x una variable. La relación de convertibilidad $=_\beta$ también se puede definir empleando reglas de inferencia:

- Axioma β

$$\frac{}{(\lambda x.M) N =_\beta M[x := N]} \text{ (axioma } \beta \text{)}$$

- Relación de equivalencia

$$\frac{}{M =_\beta M} \text{ (refl)}$$

$$\frac{M =_\beta N}{N =_\beta M} \text{ (sim)}$$

$$\frac{M =_\beta N \quad N =_\beta L}{M =_\beta L} \text{ (trans)}$$

- Relación compatible

$$\frac{M =_\beta N}{M L =_\beta N L} \text{ (appd)}$$

$$\frac{M =_\beta N}{L M =_\beta L N} \text{ (appi)}$$

$$\frac{M =_\beta N}{\lambda x.M =_\beta \lambda x.N} \text{ (abst)}$$

La teoría λ

Notación

Si $M =_{\beta} N$ es un teorema, éste es denotado por $\lambda \vdash M =_{\beta} N$.

La teoría λ

Notación

Si $M =_{\beta} N$ es un teorema, éste es denotado por $\lambda \vdash M =_{\beta} N$.

Definición 2.1.5

Los λ -términos M y N son β -**convertibles** sii $\lambda \vdash M =_{\beta} N$.

La teoría λ

Notación

Si $M =_{\beta} N$ es un teorema, éste es denotado por $\lambda \vdash M =_{\beta} N$.

Definición 2.1.5

Los λ -términos M y N son β -**convertibles** sii $\lambda \vdash M =_{\beta} N$.

Observación

La teoría λ es

- una teoría ecuacional,
- *logic-free*, es decir, las fórmulas de la teoría no contienen constantes lógicas.

La teoría λ

Ejemplo

Sean M y N dos λ -términos. Demostrar que $\lambda \vdash (\lambda x. \lambda y. x) M N =_{\beta} M$.

(i) Una demostración en forma de árbol

$$\frac{\frac{\frac{}{(\lambda x. \lambda y. x) M =_{\beta} \lambda y. M} \text{(axioma } \beta)}{(\lambda x. \lambda y. x) M N =_{\beta} (\lambda y. M) N} \text{(appd)} \quad \frac{}{(\lambda y. M) N =_{\beta} M} \text{(axioma } \beta)}{(\lambda x. \lambda y. x) M N =_{\beta} M} \text{(trans)}$$

Ejemplo

Sean M y N dos λ -términos. Demostrar que $\lambda \vdash (\lambda x. \lambda y. x) M N =_{\beta} M$.

(i) Una demostración en forma de árbol

$$\frac{\frac{\frac{}{(\lambda x. \lambda y. x) M =_{\beta} \lambda y. M} \text{(axioma } \beta)}{(\lambda x. \lambda y. x) M N =_{\beta} (\lambda y. M) N} \text{(appd)} \quad \frac{}{(\lambda y. M) N =_{\beta} M} \text{(axioma } \beta)}{(\lambda x. \lambda y. x) M N =_{\beta} M} \text{(trans)}$$

(ii) Una demostración lineal

1. $(\lambda x. \lambda y. x) M =_{\beta} \lambda y. M$ (axioma β)
2. $(\lambda x. \lambda y. x) M N =_{\beta} (\lambda y. M) N$ (appd en 1)
3. $(\lambda y. M) N =_{\beta} M$ (axioma β)
4. $(\lambda x. \lambda y. x) M N =_{\beta} M$ (trans en 2 y 3)

La teoría λ

Razonamiento ecuacional

Desde que la teoría λ es una teoría ecuacional, podemos emplear «razonamiento ecuacional» para la demostración de sus teoremas. Es decir, podemos construir cadenas de β -conversiones y sustituir λ -términos β -convertibles.

Ejemplo

Sean M y N dos λ -términos. Demostramos empleando razonamiento ecuacional que $\lambda \vdash (\lambda x. \lambda y. x) M N =_{\beta} M$.

La teoría λ

Ejemplo

Sean M y N dos λ -términos. Demostramos empleando razonamiento ecuacional que $\lambda \vdash (\lambda x. \lambda y. x) M N =_{\beta} M$.

Demostración.

$$\begin{aligned} \underline{(\lambda x. \lambda y. x) M N} &=_{\beta} \underline{(\lambda y. M) N} \\ &=_{\beta} M \end{aligned}$$

La teoría λ

Observación

Church empleó el nombre «conv» para denotar la relación de convertibilidad (véase, *p. ej.* (Church 1951)). El texto guía usa el símbolo de la igualdad «=», tal vez siguiendo a (Curry y Feys 1958, § 3.D.3). Nosotros usamos el símbolo « $=_{\beta}$ » siguiendo a (Hindley y Seldin 2008).

Tema

- Introducción
- Descripción formal del lambda cálculo
- El lambda cálculo como una teoría formal
- **Combinadores**
- Combinadores de punto fijo
- Codificación de Church
- Codificación de Barendregt
- Representación de las funciones computables
- Referencias

Combinadores

Definición 2.1.4.(ii)

Un **combinador** (o λ -**término cerrado**) es un λ -término sin variables libres.

Combinadores

Ejemplo 2.1.6 (algunos combinadores comunes)

$$I \equiv \lambda x. x$$

(identidad)

$$K \equiv \lambda x. \lambda y. x$$

(proyección primera componente)

$$K_* \equiv \lambda x. \lambda y. y$$

(proyección segunda componente)

$$S \equiv \lambda f. \lambda g. \lambda x. f x (g x)$$

(composición fuerte)

Combinadores

Teorema

Sean L, M, N λ -términos, entonces

$$\lambda \vdash \mathbf{I} M =_{\beta} M,$$

$$\lambda \vdash \mathbf{K} M N =_{\beta} M,$$

$$\lambda \vdash \mathbf{K}_* M N =_{\beta} N,$$

$$\lambda \vdash \mathbf{S} M N L =_{\beta} M L (N L).$$

Combinadores

Observación

Los programas en un lenguaje de programación basado en el lambda cálculo son combinadores. combinators.

Observación

Los combinadores **K** y **S** (es decir, la lógica combinatoria) son un lenguaje Turing-completo.

Tema

- Introducción
- Descripción formal del lambda cálculo
- El lambda cálculo como una teoría formal
- Combinadores
- **Combinadores de punto fijo**
- Codificación de Church
- Codificación de Barendregt
- Representación de las funciones computables
- Referencias

Combinadores de punto fijo

Teorema 2.1.7.(i) (teorema de punto fijo)

Para cualquier λ -término F , existe un λ -término X , tal que

$$\lambda \vdash F X =_{\beta} X.$$

Combinadores de punto fijo

Teorema 2.1.7.(i) (teorema de punto fijo)

Para cualquier λ -término F , existe un λ -término X , tal que

$$\lambda \vdash F X =_{\beta} X.$$

Demostración.

Sea $W \equiv \lambda x. F (x x)$ y sea $X \equiv W W$. Entonces,

$$\begin{aligned} X &\equiv W W \\ &\equiv (\lambda x. F (x x)) W \\ &=_{\beta} F (W W) \\ &\equiv F X. \end{aligned}$$

Es decir, $\lambda \vdash F X =_{\beta} X$. 

Combinadores de punto fijo

Definición

Un **combinador de punto fijo** es un combinador **fix** tal que, para cualquier λ -término F ,

$$\lambda \vdash \text{fix } F =_{\beta} F (\text{fix } F).$$

Combinadores de punto fijo

Teorema 2.1.7.(ii)

El combinador $Y \equiv \lambda f. V V$, donde $V \equiv \lambda x. f (x x)$, es un combinador de punto fijo.

Combinadores de punto fijo

Teorema 2.1.7.(ii)

El combinador $Y \equiv \lambda f. V V$, donde $V \equiv \lambda x. f (x x)$, es un combinador de punto fijo.

Demostración.

Debemos demostrar que $Y F =_{\beta} F (Y F)$.

$$\begin{aligned} Y F &\equiv (\lambda f. V V) F \\ &=_{\beta} (\lambda x. F (x x)) (\lambda x. F (x x)) \\ &=_{\beta} F ((\lambda x. F (x x)) (\lambda x. F (x x))) \\ &=_{\beta} F ((\lambda f. V V) F) \\ &\equiv F (Y F) \end{aligned}$$



Combinadores de punto fijo

Teorema 2.1.7.(ii)

El combinador $Y \equiv \lambda f. V V$, donde $V \equiv \lambda x. f (x x)$, es un combinador de punto fijo.^a

Demostración.

Debemos demostrar que $Y F =_{\beta} F (Y F)$.

$$\begin{aligned} Y F &\equiv (\lambda f. V V) F \\ &=_{\beta} (\lambda x. F (x x)) (\lambda x. F (x x)) \\ &=_{\beta} F ((\lambda x. F (x x)) (\lambda x. F (x x))) \\ &=_{\beta} F ((\lambda f. V V) F) \\ &\equiv F (Y F) \end{aligned}$$

^aDe acuerdo a (Hindley y Seldin 2008, pág. 36), este combinador fue insinuado por Curry en 1929 y publicado por primera vez por Rosenbloom (1950). Véase también (Barendregt 2004, Corollary 6.1.3).

Combinadores de punto fijo

Teorema

El combinador UU , donde $U \equiv \lambda u. \lambda x. x (u u x)$, es un combinador de punto fijo.^a

^aEste combinador fue definido por Turing (1937). Véase también (Barendregt 2004, Definition 6.1.4).

Recursividad usando combinadores de punto fijo

Descripción

Los combinadores de punto fijo permiten representar la recursividad en el lambda cálculo.

Recursividad usando combinadores de punto fijo

Ejemplo

Un ejemplo informal usando un combinador de punto fijo **fix** para definir la función factorial (Peyton Jones 1987, § 2.4.1).

$\text{fac} \equiv \lambda n. \text{if } (n == 0) \text{ then } 1 \text{ else } n \times \text{fac } (n - 1)$	(combinador)
$\equiv \lambda n. (\dots \text{fac } \dots)$	(combinador recursivo)
$\equiv (\lambda f. \lambda n. (\dots f \dots)) \text{fac}$	(λ -abstracción en fac)

(continua en la próxima diapositiva)

Recursividad usando combinadores de punto fijo

Ejemplo (continuación)

Ahora, sea h el combinador no recursivo

$$h \equiv \lambda f. \lambda n. (\dots f \dots),$$

y observamos que fac es un punto fijo de h

$$fac \equiv h\ fac.$$

Entonces, podemos definir fac empleando un combinador de punto fijo fix

$$fac \equiv fix\ h.$$

(continua en la próxima diapositiva)

Recursividad usando combinadores de punto fijo

Ejemplo (continuación)

$$\begin{aligned}\text{fac } 1 &\equiv \text{fix } h \ 1 \\ &=_{\beta} h \ (\text{fix } h) \ 1 \\ &\equiv (\lambda f. \lambda n. (\dots f \dots)) \ (\text{fix } h) \ 1 \\ &=_{\beta} \text{if } (1 == 0) \text{ then } 1 \text{ else } 1 \times (\text{fix } h \ 0) \\ &=_{\beta} 1 \times (\text{fix } h \ 0) \\ &=_{\beta} 1 \times (h \ (\text{fix } h) \ 0) \\ &\equiv 1 \times ((\lambda f. \lambda n. (\dots f \dots)) \ (\text{fix } h) \ 0) \\ &=_{\beta} 1 \times (\text{if } (0 == 0) \text{ then } 1 \text{ else } 1 * (\text{fix } h \ (-1))) \\ &=_{\beta} 1 \times 1 \\ &=_{\beta} 1\end{aligned}$$

Tema

- Introducción
- Descripción formal del lambda cálculo
- El lambda cálculo como una teoría formal
- Combinadores
- Combinadores de punto fijo
- **Codificación de Church**
- Codificación de Barendregt
- Representación de las funciones computables
- Referencias

Codificación de Church

Definición 2.1.9.(i)

Sean F y M dos λ -términos y sea $n \in \mathbb{N}$. Inductivamente definimos $F^n(M)$ por:

$$\begin{aligned} F^0(M) &\equiv M, \\ F^{n+1}(M) &\equiv F(F^n(M)). \end{aligned}$$

Codificación de Church

Definición

Los **numerales de Church**, denotados por c_n con $n \in \mathbb{N}$, codifican los números naturales:

$$c_n \equiv \lambda f. \lambda x. f^n(x).$$

Codificación de Church

Ejemplo

Algunos numerales.

$$c_0 \equiv \lambda f. \lambda x. x,$$

$$c_1 \equiv \lambda f. \lambda x. f x,$$

$$c_2 \equiv \lambda f. \lambda x. f (f x),$$

$$c_3 \equiv \lambda f. \lambda x. f (f (f x)).$$

Codificación de Church

Sucesor

Un combinador para el sucesor es definido por

$$\text{succ} \equiv \lambda n f x. f (n f x),$$

donde para todo $n \in \mathbb{N}$,

$$\lambda \vdash \text{succ } c_n =_{\beta} c_{n+1}.$$

Codificación de Church

Proposición 2.1.10 (adición, multiplicación y exponenciación)

Sean

$$\text{add} \equiv \lambda x. \lambda y. \lambda p. \lambda q. x p (y p q),$$

$$\text{mult} \equiv \lambda x. \lambda y. \lambda z. x (y z),$$

$$\text{exp} \equiv \lambda x. \lambda y. y x,$$

entonces para todo $m, n \in \mathbb{N}$,

$$\lambda \vdash \text{add } c_m c_n =_{\beta} c_{m+n}.$$

$$\lambda \vdash \text{mult } c_m c_n =_{\beta} c_{m \times n}.$$

$$\lambda \vdash \text{exp } c_m c_n =_{\beta} c_{m^n}, \text{ excepto para } n = 0.$$

Codificación de Church

Definición 2.2.1

Codificamos los booleanos por

$$\text{true} \equiv K, \quad \text{false} \equiv K_*,$$

donde para todos $M, N \in \Lambda$,

$$\lambda \vdash \text{true } M N =_{\beta} M,$$

$$\lambda \vdash \text{false } M N =_{\beta} N.$$

Codificación de Church

Test para cero

Un combinador para un test para cero es definido por

$$\text{isZero} \equiv \lambda n. n (\lambda x. \text{false}) \text{true},$$

donde para todo $n \in \mathbb{N}$,

$$\lambda \vdash \text{isZero } c_0 =_{\beta} \text{true},$$

$$\lambda \vdash \text{isZero } c_{n+1} =_{\beta} \text{false}.$$

Codificación de Church

Predecesor

Un combinador para el predecesor es definido por

$$\text{pred} \equiv \lambda n. \lambda f. \lambda x. n (\lambda g. \lambda h. h (g f)) (\lambda u. x) (\lambda u. u),$$

donde para todo $n \in \mathbb{N}$,

$$\lambda \vdash \text{pred } c_0 =_{\beta} c_0,$$

$$\lambda \vdash \text{pred } c_{n+1} =_{\beta} c_n.$$

Tema

- Introducción
- Descripción formal del lambda cálculo
- El lambda cálculo como una teoría formal
- Combinadores
- Combinadores de punto fijo
- Codificación de Church
- **Codificación de Barendregt**
- Representación de las funciones computables
- Referencias

Codificación de Barendregt

Definición 2.2.1

Codificamos los booleanos por

$$\text{true} \equiv K, \quad \text{false} \equiv K_*,$$

donde para todos $M, N \in \Lambda$,

$$\lambda \vdash \text{true } M N =_\beta M,$$

$$\lambda \vdash \text{false } M N =_\beta N.$$

Codificación de Barendregt

Definición 2.2.2

Sean M y N dos λ -términos. Codificamos un par ordenado de M y N por:

$$[M, N] \equiv \lambda z. z M N,$$

$$\text{fst} \equiv \lambda p. p \text{ true},$$

$$\text{snd} \equiv \lambda p. p \text{ false},$$

donde para todos $M, N \in \Lambda$,

$$\lambda \vdash \text{fst} [M, N] =_{\beta} M,$$

$$\lambda \vdash \text{snd} [M, N] =_{\beta} N.$$

Codificación de Barendregt

Definición 2.2.2

Sean M y N dos λ -términos. Codificamos un par ordenado de M y N por:^a

$$[M, N] \equiv \lambda z. z M N,$$

$$\text{fst} \equiv \lambda p. p \text{ true},$$

$$\text{snd} \equiv \lambda p. p \text{ false},$$

donde para todos $M, N \in \Lambda$,

$$\lambda \vdash \text{fst} [M, N] =_{\beta} M,$$

$$\lambda \vdash \text{snd} [M, N] =_{\beta} N.$$

^aEl texto guía no usa un nombre para el combinador fst ni para el combinador snd .

Codificación de Barendregt

Definición 2.2.3

Los **numerales de Barendregt**, denotados por $\overline{n}$, con $n \in \mathbb{N}$, codifican los números naturales:

$$\begin{aligned}\overline{0} &\equiv \mathbf{I}, \\ \overline{n+1} &\equiv [\mathbf{false}, \overline{n}].\end{aligned}$$

Codificación de Barendregt

Lema 2.2.4 (sucesor, predecesor y test para cero)

Sean

$$S^+ \equiv \lambda x. [\text{false}, x], \quad P^- \equiv \lambda x. x \text{ false}, \quad \text{Zero} \equiv \lambda x. x \text{ true},$$

entonces para todo $n \in \mathbb{N}$,

$$\lambda \vdash S^+ \bar{n} =_{\beta} \overline{n+1},$$

$$\lambda \vdash P^- \bar{0} =_{\beta} \text{false},$$

$$\lambda \vdash P^- \overline{n+1} =_{\beta} \bar{n},$$

$$\lambda \vdash \text{Zero} \bar{0} =_{\beta} \text{true},$$

$$\lambda \vdash \text{Zero} \overline{n+1} =_{\beta} \text{false}.$$

Tema

- Introducción
- Descripción formal del lambda cálculo
- El lambda cálculo como una teoría formal
- Combinadores
- Combinadores de punto fijo
- Codificación de Church
- Codificación de Barendregt
- **Representación de las funciones computables**
- Referencias

Representación de las funciones computables

Definición

Una **función numérico-teórica** es una función

$$\mathbb{N}^p \rightarrow \mathbb{N}, \text{ con } p \in \mathbb{N}.$$

Representación de las funciones computables

Definición 2.2.5

Sea $f : \mathbb{N}^p \rightarrow \mathbb{N}$ una función numérico-teórica. La función f es λ -definible respecto a la codificación de Barendregt sii existe un combinador F , tal que para todo $n_1, \dots, n_p \in \mathbb{N}$,

$$\lambda \vdash F \overline{n_1} \dots \overline{n_p} =_{\beta} \overline{f(n_1, \dots, n_p)}.$$

Representación de las funciones computables

Lema 2.2.9

Las funciones iniciales son λ -definibles (respecto a la codificación de Barendregt).

Lema 2.2.10

Las funciones λ -definibles (respecto a la codificación de Barendregt) son cerradas bajo composición.

Lema 2.2.11

Las funciones λ -definibles (respecto a la codificación de Barendregt) son cerradas bajo recursión primitiva.

Lema 2.2.12

Las funciones λ -definibles (respecto a la codificación de Barendregt) son cerradas bajo minimalización.

Representación de las funciones computables

Teorema 2.2.13

Las funciones recursivas son λ -definibles (respecto a la codificación de Barendregt).

Representación de las funciones computables

Definición 2.2.5

Sea $f : \mathbb{N}^p \rightarrow \mathbb{N}$ una función numérico-teórica. La función f es **λ -definible respecto a la codificación de Church** sii existe un combinador F , tal que para todo $n_1, \dots, n_p \in \mathbb{N}$,

$$\lambda \vdash F c_{n_1} \dots c_{n_p} =_{\beta} c_{f(n_1, \dots, n_p)}.$$

Representación de las funciones computables

Teorema 2.2.14

Las funciones recursivas son λ -definibles (respecto a la codificación de Church).

Tema

- Introducción
- Descripción formal del lambda cálculo
- El lambda cálculo como una teoría formal
- Combinadores
- Combinadores de punto fijo
- Codificación de Church
- Codificación de Barendregt
- Representación de las funciones computables
- Referencias

Referencias

-  H. P. Barendregt (2004). The Lambda Calculus. Its Syntax and Semantics. Revised edition, 6th impression. Vol. 103. Studies in Logic and the Foundations of Mathematics. Elsevier, 2004 (1981) (vid. págs. [10](#), [34-38](#), [43](#), [67](#), [68](#)).
-  Henk Barendregt (1992a). Functional Programming and Lambda Calculus. En: Handbook of Theoretical Computer Science. Volume B. Formal Models and Semantics. Ed. por J. van Leeuwen. Second impression. MIT Press, 1992 (1990). Cap. 7. DOI: [10.1016/B978-0-444-88074-1.50012-3](#) (vid. [pág. 3](#)).
-  — (1992b). Lambda Calculi with Types. En: Handbook of Logic in Computer Science. Volume 2. Ed. por S. Abramsky, Dov M. Gabbay y T. S. E. Maibaum. Clarendon Press, 1992, págs. 117-309 (vid. [pág. 10](#)).
-  Henk Barendregt y Erik Barendsen (2000). Introduction to Lambda Calculus. Revisited edition. 2000 (vid. [pág. 10](#)).
-  Felice Cardone y J. Roger Hindley (2009). Lambda-Calculus and Combinators in the 20th Century. En: Handbook of Logic in Computer Science. Ed. por Dov M. Gabbay y John Woods. Vol. 5. Elsevier, 2009, págs. 723-817 (vid. [pág. 10](#)).

Referencias

-  Alonzo Church (1951). The Calculi of Lambda-Conversion. Second printing. Princeton University Press, 1951 (1941) (vid. pág. 55).
-  Haskell B. Curry y Robert Feys (1958). Combinatory Logic. Vol. I. North-Holland Publishing Company, 1958 (vid. pág. 55).
-  Dov M. Gabbay y John Woods, eds. (2009). Handbook of the History of Logic. Vol. 5. Elsevier, 2009.
-  J. Roger Hindley y Jonathan P. Seldin (2008). Lambda-Calculus and Combinators. An Introduction. Cambridge University Press, 2008 (vid. págs. 10, 55, 67).
-  Simon L. Peyton Jones (1987). The Implementation of Functional Programming Languages. Series in Computer Sciences. Prentice-Hall International, 1987 (vid. pág. 70).
-  Benjamin C. Pierce (2002). Types and Programming Languages. MIT Press, 2002 (vid. pág. 22).
-  Paul C. Rosenbloom (1950). The Elements of Mathematical Logic. Dover Publications, 1950 (vid. pág. 67).

Referencias

-  J. Barkley Rosser (1984). Highlights of the History of Lambda-Calculus. Annals of the History of Computing 6.4, págs. 337-349. DOI: [10.1109/MAHC.1984.10040](https://doi.org/10.1109/MAHC.1984.10040) (vid. pág. 10).
-  Jonathan P. Seldin (2009). The Logic of Church and Curry. En: Handbook of Logic in Computer Science. Ed. por Dov M. Gabbay y John Woods. Vol. 5. Elsevier, 2009, págs. 819-873 (vid. pág. 10).
-  A. M. Turing (1937). The μ -Function in λ -K-Conversion. The Journal of Symbolic Logic 4.2, pág. 164. DOI: [10.2307/2268281](https://doi.org/10.2307/2268281) (vid. pág. 68).